

Spotify Like Classifier (TensorFlow/Keras)

Below is a complete solution for all **5 tasks**. Save the code in a file named:

spotify_like_classifier.py

Step 1: Install TensorFlow & Keras

Open Terminal or Command Prompt:

```
pip install tensorflow numpy
```

Verify installation:

```
import tensorflow as tf
```

```
print(tf.__version__)
```

Step 2–5: spotify_like_classifier.py

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import EarlyStopping
```

```
# =====
# Generate Random Dataset
# =====
```

```
np.random.seed(42)
```

```
# 100 samples with 2 features:
# Feature 1: Listening Time
# Feature 2: Liked Genre Score
X = np.random.rand(100, 2)
```

```
# Binary Labels (0 or 1)
y = np.random.randint(0, 2, size=(100,))
```

```
# =====
# Build Sequential Model
# =====
```

```
model = Sequential([
```

```

Dense(
    16,
    activation='relu',
    input_shape=(2,)
),

Dense(
    8,
    activation='relu'
),

Dense(
    1,
    activation='sigmoid'
)

])

# =====
# Compile Model
# =====

model.compile(

    optimizer='adam',

    loss='binary_crossentropy',

    metrics=['accuracy']

)

# Print Model Summary
print("\nModel Summary:\n")
model.summary()

# =====
# Early Stopping Callback
# =====

early_stop = EarlyStopping(

```

```

        monitor='val_loss',

        patience=3,

        restore_best_weights=True

    )

# =====
# Train Model
# =====

history = model.fit(
    X,
    y,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)

# =====
# Print Final Accuracy
# =====

print("\nFinal Training Accuracy:",
      history.history['accuracy'][-1])

print("Final Validation Accuracy:",
      history.history['val_accuracy'][-1])

```

Explanation of Each Task

Task 1: Install TensorFlow & Create File

Install the required packages:

pip install tensorflow numpy

Create a new Python file:

spotify_like_classifier.py

Task 2: Build the Neural Network

The model has **three Dense layers**:

Input (2 Features)



Dense (16, ReLU)



Dense (8, ReLU)



Dense (1, Sigmoid)

- **Input features:**
 - listening_time
 - liked_genre_score
 - **Output:** Probability that the user likes the song.
-

Task 3: Compile the Model

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

- **Optimizer:** Adam
 - **Loss Function:** Binary Cross-Entropy (for binary classification)
 - **Metric:** Accuracy
-

Task 4: Train with Random Data

The script creates:

```
X = np.random.rand(100,2)
```

This generates **100 user profiles**.

Example:

Listening Time Genre Score

0.62 0.34

0.90 0.75

0.11 0.54

Binary labels:

```
y = np.random.randint(0,2,100)
```

Example:

```
1
0
1
1
0
...
```

The model is trained for **20 epochs** (unless stopped early):

```
model.fit(...)
```

Because the data is random, the accuracy will also be essentially random (often around **50–70%**) and is **not meaningful**. In a real project, you would train on real labeled Spotify user data.

Task 5: Early Stopping

```
EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
```

How it works:

- Watches the **validation loss** after each epoch.
- If it does **not improve for 3 consecutive epochs**, training stops automatically.
- `restore_best_weights=True` reloads the model weights from the epoch with the lowest validation loss.

Benefits:

- Prevents overfitting.

- Saves training time.
- Keeps the best-performing model.

Expected Output

Model: "sequential"

Layer (type)	Output Shape	Param #
Dense	(None,16)	
Dense	(None,8)	
Dense	(None,1)	

=====

Epoch 1/20

...

Epoch 8/20

...

Training stopped early

Final Training Accuracy: 0.68

Final Validation Accuracy: 0.60